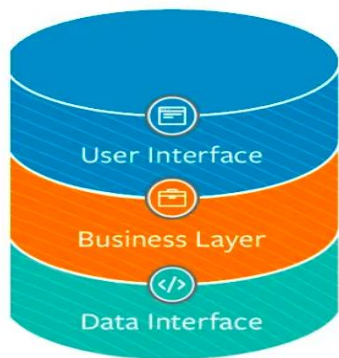




Лекція 20

Мікросервісна архітектура

Spring Cloud



Монолітна архітектура — це традиційна архітектура програмного забезпечення, яка об'єднує всі компоненти програми в єдиний неподільний блок, що полегшує її розробку та розгортання.

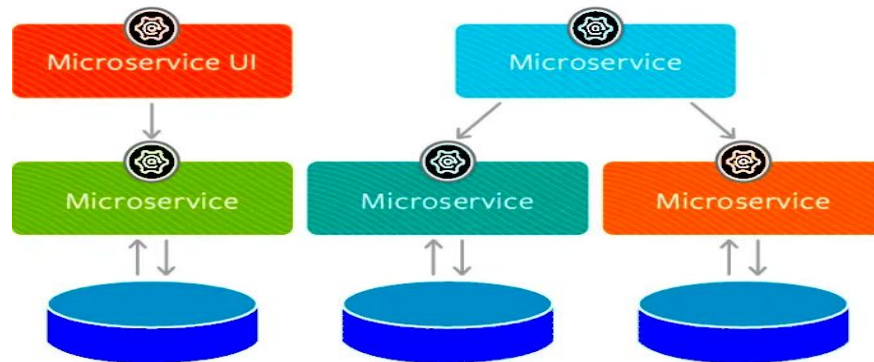
Переваги

- **Простота для розуміння:** монолітну архітектуру легше зрозуміти та розробити
- **Просте розгортання:** оскільки всі компоненти упаковані разом, розгортання легше та швидше
- **Краща продуктивність:** компоненти можуть взаємодіяти один з одним ефективніше
- **Простіше тестування:** тестується лише одна кодова база.
- **Простіше дебажити:** коли виникає помилка, легше визначити першопричину в монолітній архітектурі, оскільки всі компоненти містяться в одній кодовій базі
- **Простіше масштабування:** горизонтальне масштабування шляхом запуску кількох копій за балансувальником навантаження

Недоліки монолітної архітектури

- **Складність:** у міру зростання проекту кодова база може ставати все складнішою і складнішою в управлінні, що з годом призводить до накопичення технічного боргу
- **Обмежена масштабованість:** немає можливості масштабувати програмні компоненти незалежно
- **Обмежена гнучкість через високу зв'язність компонентів:** будь-яка зміна коду впливає на всю систему, тому вона створює більше технічних проблем і потребує ретельної координації
- **Більш тривалі цикли розробки:** розробка та розгортання змін у монолітному додатку може зайняти більше часу через необхідність тестування та розгортання всього додатка
- **Вищий ризик внесення змін:** зміни до монолітної програми можуть бути ризикованими, оскільки невелика зміна в одному місці може мати несподівані наслідки в інших частинах програми
- **Обмежений вибір технологій:** у монолітній архітектурі всі компоненти повинні використовувати той самий технологічний стек
- **Складність впровадження нових технологій:** оскільки всі компоненти тісно інтегровані, впровадження нових технологій у монолітній архітектурі без істотних змін усієї програми може виявитися складним завданням

Мікросервісна архітектура



Переваги

- **Масштабування:** можна масштабувати кожен мікросервіс відповідно до його конкретних потреб, не впливаючи на інші частини програми.
- **Стійкість:** якщо один мікросервіс виходить з ладу, інші мікросервіси можуть продовжувати працювати, мінімізуючи вплив на загальну програму.
- **Гнучкість:** можна розробляти, розгортати та керувати сервісами незалежно один від одного.
- **Більш швидкий цикл розробки:** мікросервіси дозволяють командам працювати більш незалежно та робити більш швидкі ітерації між релізами.
- **Різноманітність технологій:** мікросервіси можна створювати за допомогою різних технологій і мов програмування, що дозволяє розробникам використовувати найкращий інструмент для роботи.
- **Невеликі цілеспрямовані команди:** великі команди, як правило, менш продуктивні, оскільки спілкування відбувається повільніше, накладні витрати на управління збільшуються, а гнучкість зменшується.
- **Проста підтримка:** оскільки мікросервіси менші та більш модульні, ніж монолітні програми, їх, як правило, легше підтримувати та оновлювати з часом.

Недоліки мікросервісної архітектури

- **Підвищена складність:** розробка, розгортання та керування декількома незалежними сервісами може бути більш складним завданням, ніж керування монолітною програмою
- **Управління розподіленими системами:** вимагає додаткових інструментів та інфраструктури для керування сервісами, а також зв'язку та координації між ними.
- **Більш складна реалізація наскрізної функціональності:** деяка функціональність повинна бути однаковою в усіх, або майже усіх сервісах (логування, зовнішня конфігурація, безпека, моніторинг).
- **Складність тестування:** кожен сервіс потрібно тестувати незалежно, а також у поєднанні з іншими сервісами. Це вимагає додаткових зусиль для створення та підтримки тестових середовищ
- **Розділена база даних:** бізнес-транзакції, які оновлюють кілька бізнес-сутностей у програмі на основі мікросервісів, потребують оновлення кількох баз даних, що належать різним службам

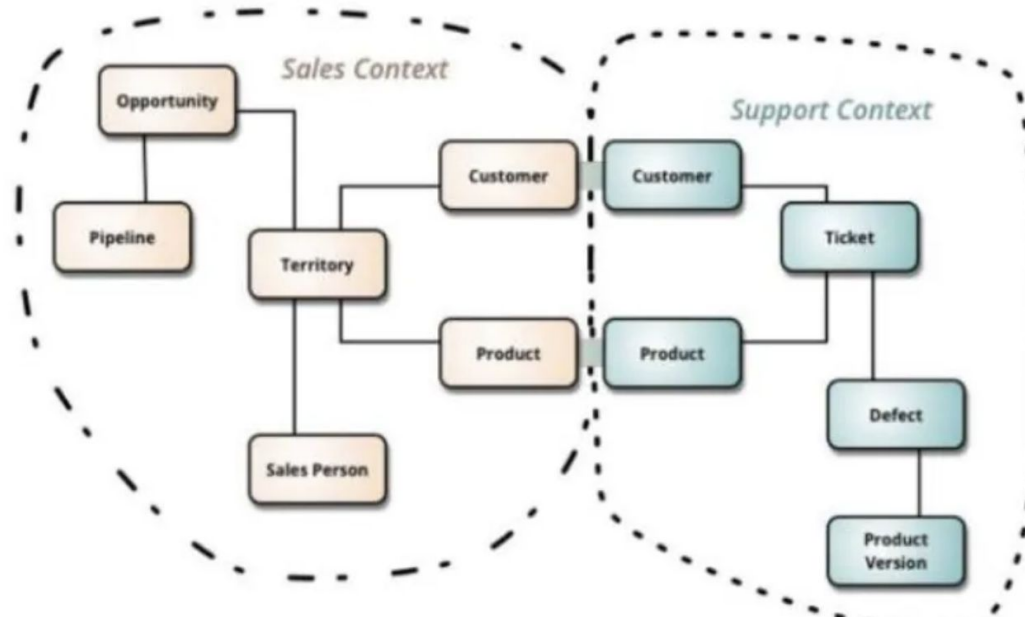
Деякі анти-паттерни мікросервісної архітектури

- **Розподілений моноліт:** мікросервіси дуже сильно пов'язані між собою, зі спільними бібліотеками та сховищами даних або з відсутністю чітких меж
- **Спільна БД:** в деяких випадках все ж таки можна використовувати спільну БД для декількох мікросервісів, але треба чітко розуміти причину для цього
- **Відсутність версіювання сервісів**
- **Мікросервіси як самоціль:** постійне бажання розбивати сервіси на все більш маленьки частини без належної мотивації
- **Розподілення на мікросервіси за технологіями:** коли мікросервіс об'єднує в собі декілька різних функцій тільки тому що вони реалізовані на одній технології
- **Надмірний функціонал в кожному сервісі:** коли кожен мікросервіс реалізує логіку спільну для всієї системи (аутентифікація, авторизація, роутинг, тощо)

Bounded context

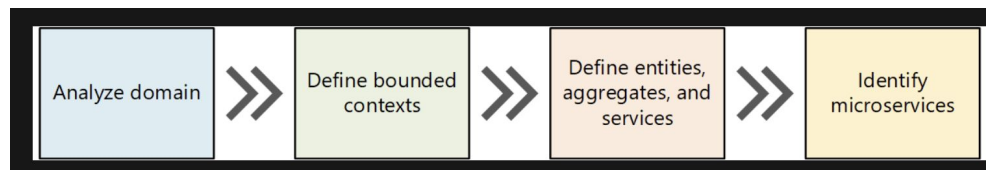
Термін використовується для опису чітких меж або поділу сервісу в більшій системі. Він визначає область дії конкретного мікросервісу та його взаємодію з іншими сервісами.

Bounded context інкапсулює внутрішній стан і логіку одного або декількох сервісів та визначає зв'язки з іншими сервісами. Кожен context може мати власний рівень зберігання, обробки та представлення даних.



Декомпозиція мікросервісів

1. Аналіз бізнес-домену, щоб зрозуміти функціональні вимоги програми
2. Визначення bounded contexts домену. Кожен context може включати субдомени.
3. Для кожного із контекстів ми визначаємо сутності, агрегати та функції (Tactical DDD)
4. Визначаються межі для кожного мікросервісу



Іноді для простоти можна поділити мікросервіси на ті, які створені для роботи з окремими сутностями (noun-based decomposition: UserService, DepartmentService) та на ті, які виконують (verb-based decomposition: AuthenticationService, EmailSenderService)

Синхронна міжсервісна взаємодія

Сервіс-клієнт викликає API, який надає інший сервіс При цьому клієнт чекає відповіді від іншого сервісу.

1. **REST через HTTP:**

- визначає єдиний інтерфейс на основі дієслів HTTP
- має чітко визначену семантику з точки зору ідемпотентності, побічних ефектів і кодів відповідей
- забезпечує stateless зв'язок, що покращує масштабованість
- підтримується майже всіма фреймворками та мовами
- не потребує додаткових бібліотек

2. **RPC (gRPC, Apache Avro, Apache Thrift):**

- більше орієнтований на операції чи команди
- підтримують бінарну серіалізацію та, як правило, ефективніші, ніж HTTP
- визначають власний Interface definition language (IDL), що може сприяти документації та тестуванню

3. **SOAP**

В цілому основним вибором має бути REST, якщо не потрібні переваги в швидкості бінарного протоколу

Асинхрона міжсервісна взаємодія

Сервіс надсилає повідомлення, не чекаючи відповіді, і один або кілька сервісів обробляють повідомлення асинхронно (Kafka, RabbitMQ)

Переваги асинхронної взаємодії:

- **reduced coupling**: відправнику повідомлення не обов'язково знати про отримувача
- **failure isolation**: якщо один з отримувачів впаде, то відправник все одно може відправляти нові повідомлення
- **responsiveness**: відправник може відповісти швидше, якщо не чекає на відповідь
- **вирівнювання навантаження**: черга повідомлень може діяти як буфер для вирівнювання робочого навантаження

Недоліки асинхронної взаємодії:

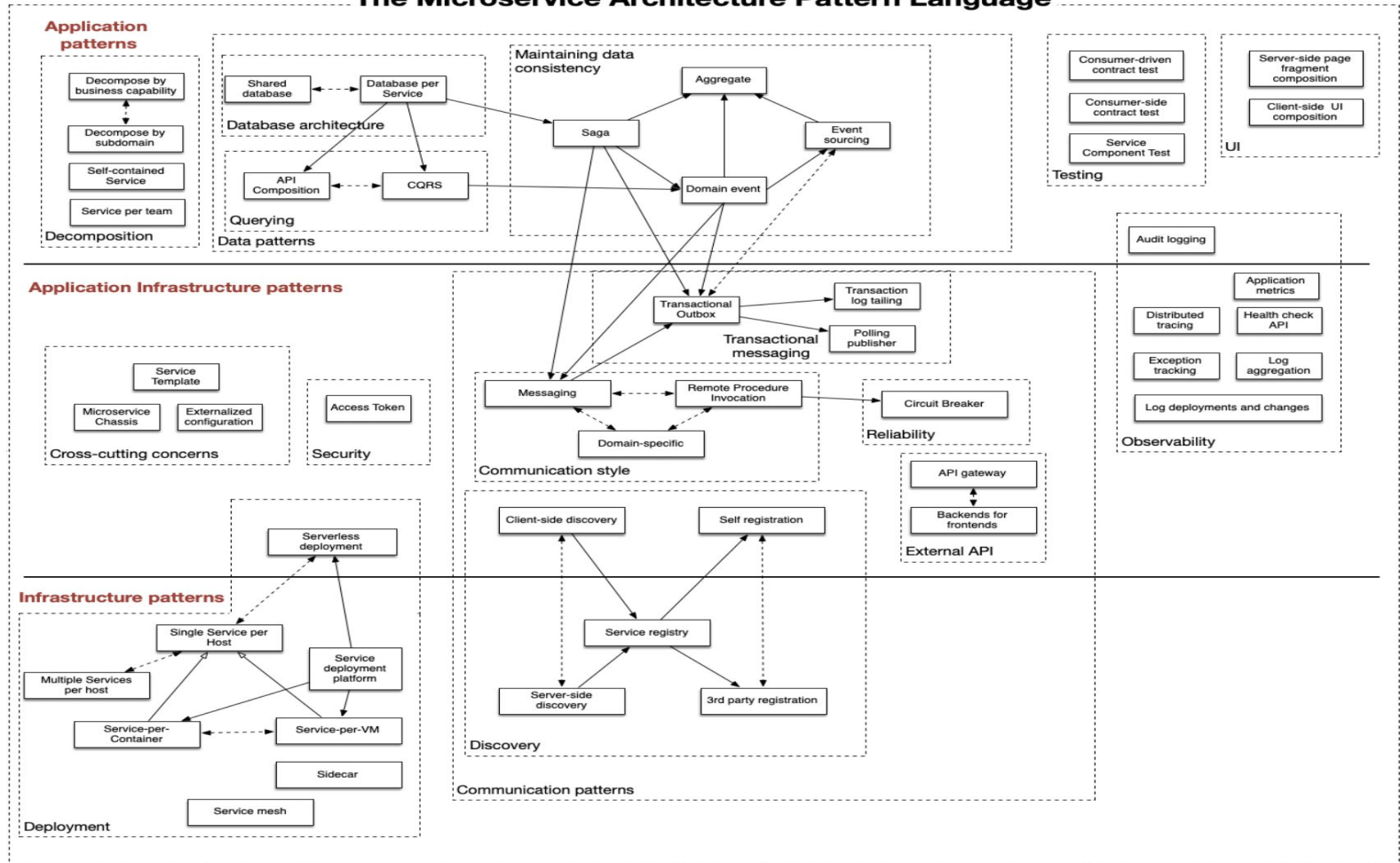
- **більша складність в реалізації** (необхідність обробки дублікатів, підтримка і налагодження додаткової інфраструктури)
- іноді модель request-response більш прийнятна з точки зору логіки додатка

Проблеми які треба вирішувати в мікросервісній архітектурі

- **Service discovery.** Оскільки архітектура мікросервісів складається з багатьох невеликих сервісів, важливо мати спосіб виявити та знайти ці сервіси. Цього можна досягти за допомогою **Service Registry** або **Service Mesh**
- **Load balancing.** Для багатьох невеликих сервісів важливо забезпечити рівномірний розподіл запитів між ними. Балансування навантаження може допомогти запобігти перевантаженню певних сервісів і забезпечити високу доступність (**client-side load balancing, server-side load-balancing, Service Mesh**)
- **Distributed tracing.** Одна транзакція може охоплювати декілька сервісів і це може ускладнити моніторинг загальної продуктивності та працездатності системи. Навіть якщо кожен сервіс генерує логи, то без гарного способу зв'язати їх разом, вони мають обмежене використання.
- **Resiliency та Fault Tolerance.** Можуть існувати десятки або навіть сотні екземплярів будь-якого мікросервісу. Екземпляр може вийти з ладу з будь-якої причини в будь-яку мить. Може виникнути збій на рівні вузла, наприклад збій апаратного забезпечення або перезавантаження віртуальної машини, або сервіс може бути переповнений запитами та бути не в змозі обробити нові запити. Будь-яка з цих подій може спричинити збій мережевого виклику (Retry, Circuit Breaker, Fallback)
- **Security.** Оскільки багато сервісів спілкуються один з одним, важливо забезпечити безпеку системи. Це включає захист зв'язку між сервісами, автентифікацію та авторизацію доступу до послуг, а також захист конфіденційних даних
- **Data consistency.** Кожен мікросервіс може мати свою БД і всі ці БД треба тримати в консистентному стані. Для того, щоб досягти цього використовуються різні паттерни та техніки: distributed transactions, eventual consistency, saga, event sourcing
- **Service Configuration.** Кожен із сервісів має свої налаштування і параметри, котрі набагато легше дивитися та змінювати їх в одному місці

Паттерни та підходи

The Microservice Architecture Pattern Language

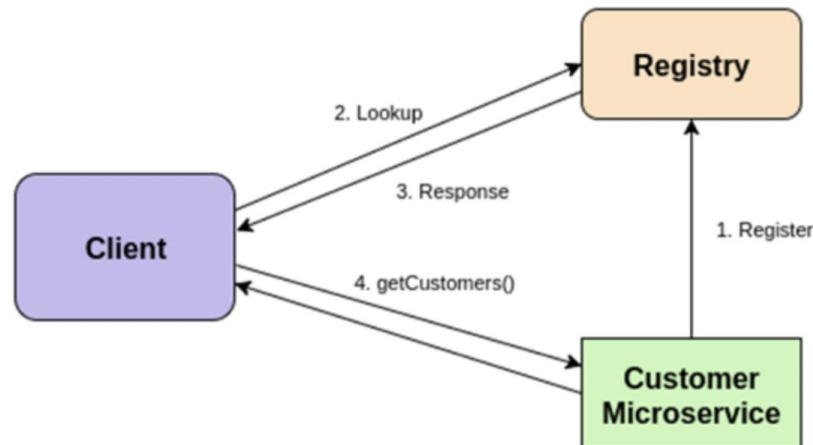


Service Registry

Це шаблон архітектури мікросервісів, який дозволяє сервісам виявляти інші мікросервіси та спілкуватися один з одним.

Коли кожен сервіс запускається, він самостійно реєструється в реєстрі, вказавши своє ім'я та адресу, за який він доступний. Після цього інші мікросервіси можуть знаходити його в реєстрі та відправляти запити до нього. Окрім реєстрації та зберігання Service Registry також періодично перевіряє доступність кожного сервісу за допомогою health-check запитів.

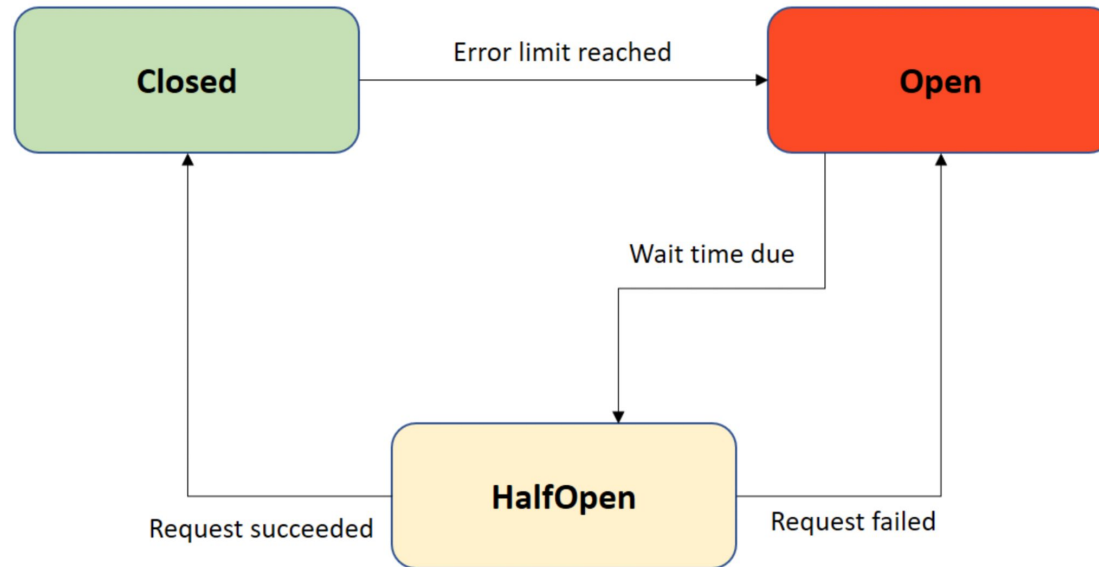
Відомі реалізації: Spring Cloud Netflix Eureka. Consul. Kubernetes



Circuit Breaker

Circuit Breaker запобігає каскадним збоям, розриваючи ланцюг, і дозволяє програмам продовжувати роботу в разі збою одного або кількох сервісів. Circuit Breaker відстежує стан сервісу і, якщо він виявляє, що сервіс не працює, він може розімкнути ланцюг і запобігти надсиланню подальших запитів до сервісу, доки він не відновиться.

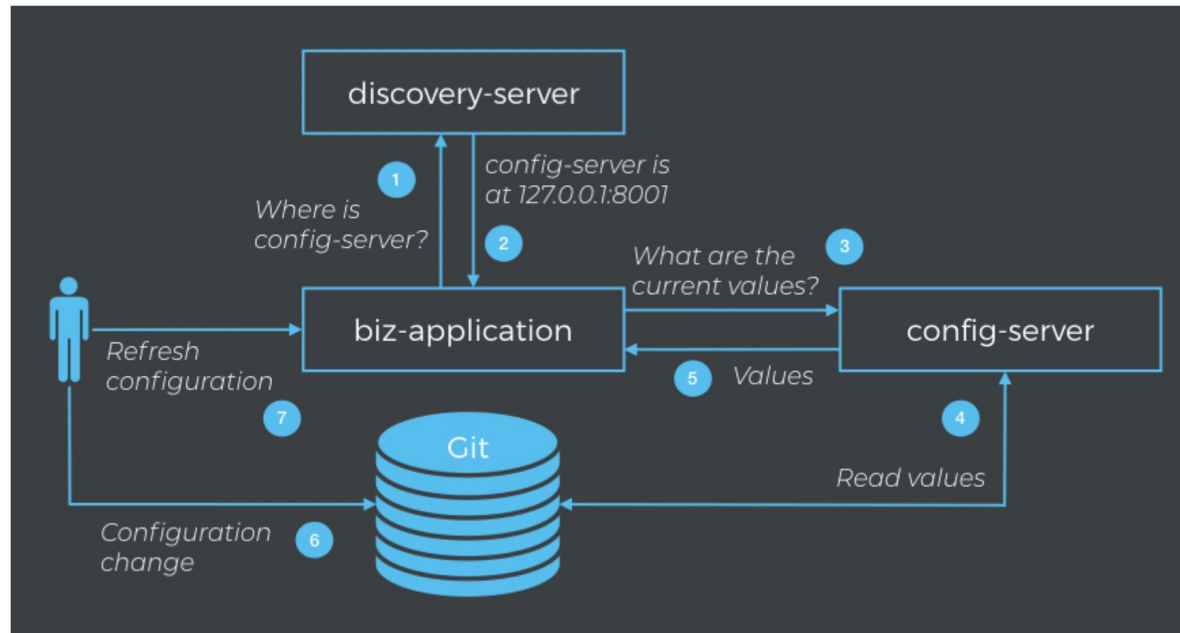
Реалізації: Netflix Hystrix, Spring Cloud Circuit Breaker



Externalized Configuration

Зберігання конфігураційних даних поза кодом програми, що дозволяє сервісу працювати в кількох середовищах без зміни основного коду. Дозволяє швидко змінювати конфігурацію для мікросервісу, який реплікований сто разів (запущено сто процесів) в одному місці.

Реалізації: Spring Cloud Config, Consul/Vault, Netflix Archaius



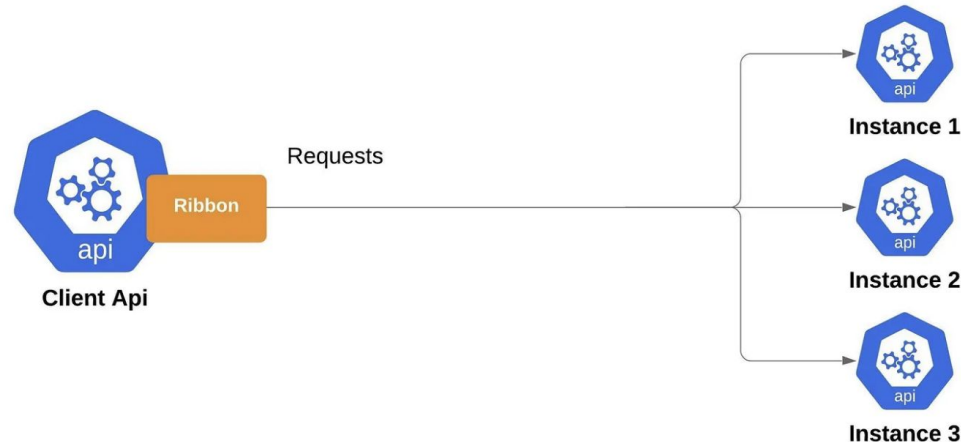
Load Balancing

Load balancing – це процес розподілу трафіку між різними екземплярами одного сервісу.

Існує два типи балансування навантаження: server-side та client-side. Різні алгоритми:

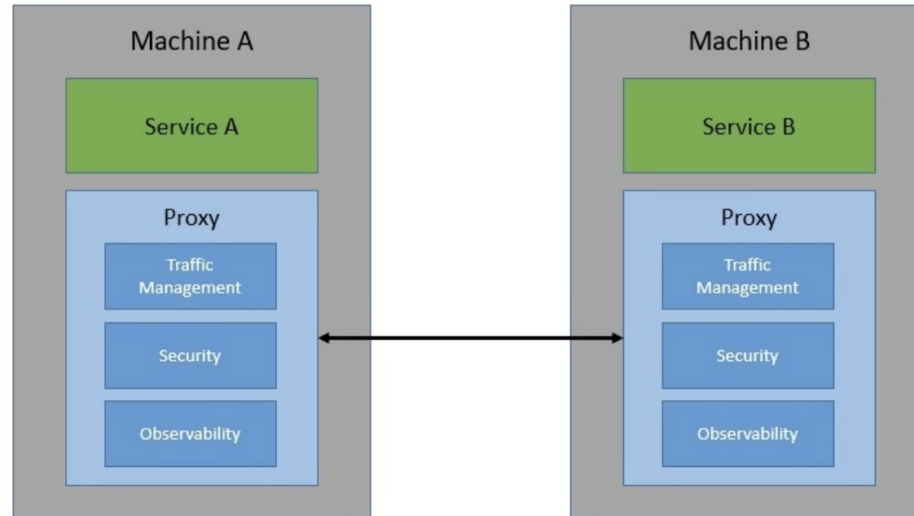
- **Random selection:** вибір екземпляра випадковим чином
- **Round-robin:** щоразу вибір екземпляра в тому самому порядку
- **Least connections:** вибір екземпляра з найменшою кількістю поточних підключень
- **Weighted metric:** використання зваженої метрики для вибору найкращого екземпляра (наприклад, використання ЦП або пам'яті)
- **IP hash:** використання хешу IP-адреси сервісу для вибору екземпляра

Реалізації client-side load balancing: Netflix Ribbon, Spring Cloud Load Balancer



Service mesh бере на себе відповідальність за керування всіма зв'язками між сервісами в рамках розподіленої програмної системи. Service mesh робить це через масив мережевих проксі, через котрі проходять всі запити між сервісами і які працюють разом із сервісами, але знаходяться на рівні інфраструктури.

Реалізації: Istio, EaseMesh, Linkerd, Consul Connect, Kubernetes

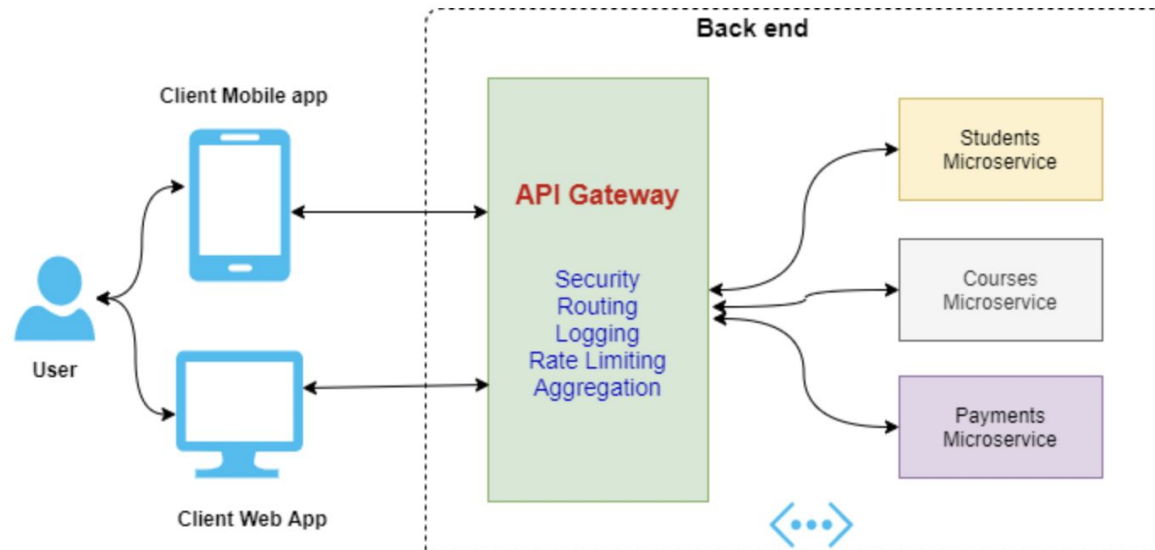


API Gateway

Основна мета **API Gateway** — відокремити клієнтів від мікросервісів, абстрагуючи складність системи за спрощеним і узгодженим API. Це також означає, що клієнту не потрібно знати адреси понад 100 мікросервісів, щоб зробити запит.

API Gateway також може виконувати додаткові функції, такі як трансформування запитів і відповідей, обмеження швидкості, автентифікація та авторизація та кешування.

Реалізації: Netflix Zuul, Spring Cloud Gateway



Spring Cloud — це набір із інструментів та бібліотек для створення програм на основі мікросервісів. Побудований на основі Spring Boot і допомагає у вирішенні типових проблем у розподілених системах:

- **Service Discovery** (Consul, Netflix Eureka, Apache Zookeeper, Kubernetes Service Discovery)
- **Load Balancing** (Spring Cloud LB, Netflix Ribbon, Kubernetes Service LB)
- **Centralized config** (Spring Cloud Config, Vault, Consul, Zookeeper)
- **Feign Client** - декларативний HTTP клієнт, що полегшує REST запити між сервісами
- **API Gateway** (Spring Cloud Gateway, Netflix Zuul)
- **Fault Tolerance** (Netflix Hystrix, Resilience4J)
- **Tracing** (Spring Cloud Sleuth, Zipkin, Jaeger)

Consul Service Discovery

HashiCorp Consul is a highly available and distributed service discovery and configuration system that provides features such as health checking, load balancing, and service segmentation.

- запускається як окремий сервіс
- централізовано зберігає та реплікує налаштування потрібні для мікросервісів
- кожен сервіс при запуску реєструється у Consul, що дозволяє звертатися до нього по імені

<dependency>

<groupId>org.springframework.cloud**</groupId>**

<artifactId>spring-cloud-starter-consul-config**</artifactId>**

</dependency>

<dependency>

<groupId>org.springframework.cloud**</groupId>**

<artifactId>spring-cloud-starter-consul-discovery**</artifactId>**

</dependency>

spring.config.import: optional:consul:/

spring.cloud.consul.host: localhost

spring.cloud.consul.port: 8500

spring.cloud.consul.discovery.health-check-path: /health

<https://medium.com/velotio-perspectives/a-practical-guide-to-hashicorp-consul-part-1-5ee778a7fcf4#>

pom.xml:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

@EnableFeignClients

@SpringBootApplication

```
public class ServiceContractsApplication {
    public static void main(String[] args) {
        SpringApplication.run(ServiceContractsApplication.class, args);
    }
}
```

@FeignClient(name = "service-customers", dismiss404 = true)

```
public interface CustomerClient {

    @GetMapping(value = "/api/customers/{id}")
    Optional<CustomerDetailsDto> getCustomer(@PathVariable("id") String id);

}
```

- интегрированный service discovery
- интегрированный load balancer

<dependency>

<groupId>org.springframework.boot**</groupId>**

<artifactId>spring-boot-starter-webflux**</artifactId>**

</dependency>

<dependency>

<groupId>org.springframework.cloud**</groupId>**

<artifactId>spring-cloud-starter-gateway**</artifactId>**

</dependency>

gateway:

routes:

- **id:** service-customers-route

uri: lb://service-customers

predicates:

- Path=/api/customers/**

filters:

- RewritePath=/api/customers/(?<remaining>.*), /api/customers/\${remaining}

- **id:** service-contracts-route

<https://cloud.spring.io/spring-cloud-gateway/reference/html/>

Circuit Breaker with Resilience4j

pom.xml:

```
<dependency>
```

```
  <groupId>org.springframework.cloud</groupId>
```

```
  <artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>
```

```
</dependency>
```

application.yml

```
resilience4j:
```

```
  circuitbreaker:
```

```
    instances:
```

```
      services:
```

```
        wait-duration-in-open-state: 10s
```

```
        failure-rate-threshold: 60
```

```
        slow-call-duration-threshold:
```

```
          - 2s
```

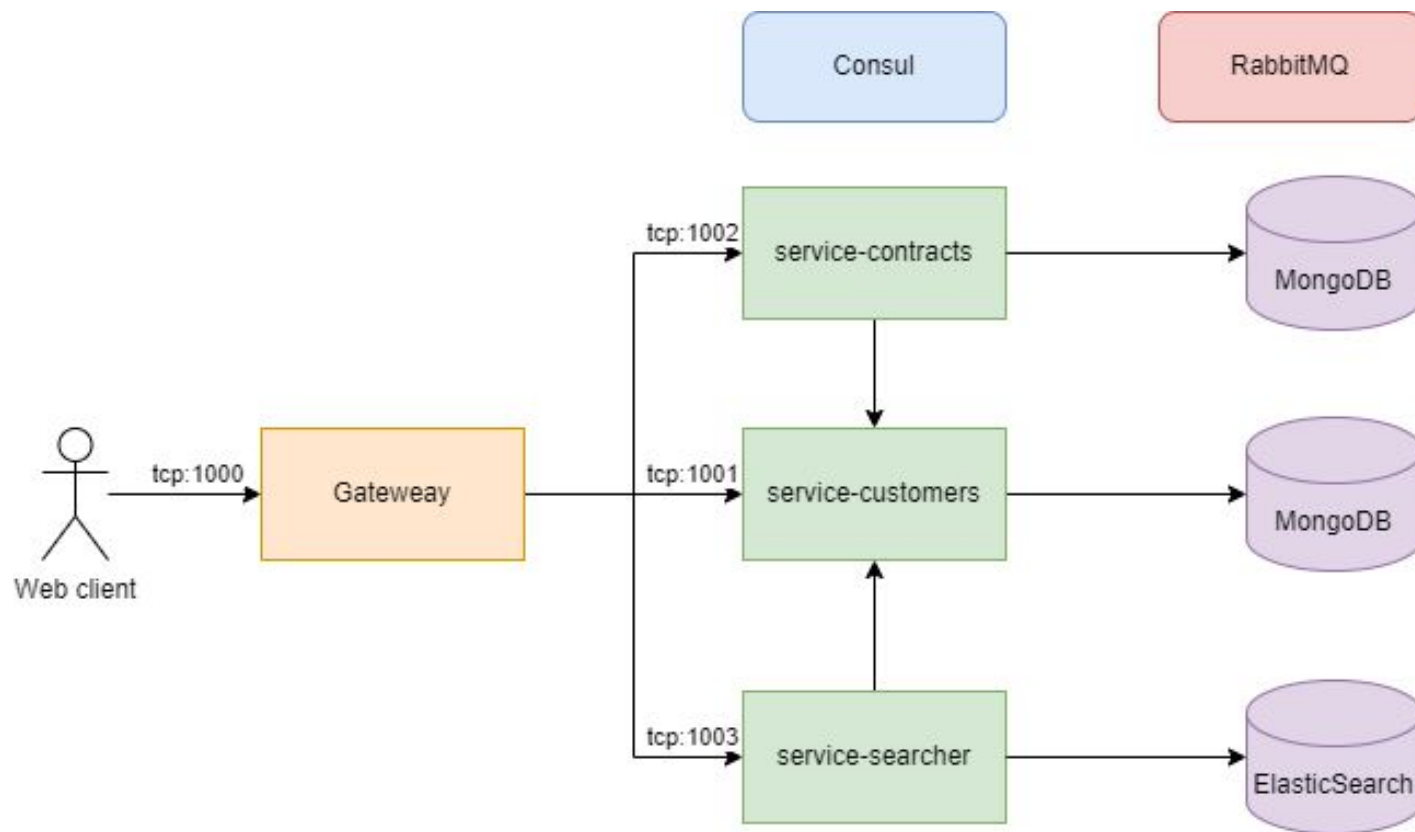
```
        minimum-number-of-calls: 5
```

```
        sliding-window-size: 3
```

```
        permitted-number-of-calls-in-half-open-state: 1
```

<https://reflectoring.io/circuitbreaker-with-resilience4j/>

Приклад



<https://github.com/profitsoft-jfd/gateway-sample>